



미니프로젝트4# Terraform

Type	프로젝트
날짜	@2025/05/26 → 2025/05/29

목차

[프로젝트 개요](#)

[사용 환경 및 소프트웨어 버전](#)

[아키텍처](#)

[AWS 리소스](#)

[프로젝트 내용](#)

[디렉토리 구조](#)

[Terraform 파일 작성](#)

[Ansible 파일 작성](#)

[자동화 스크립트 작성](#)

[배포 및 검증](#)

프로젝트 개요

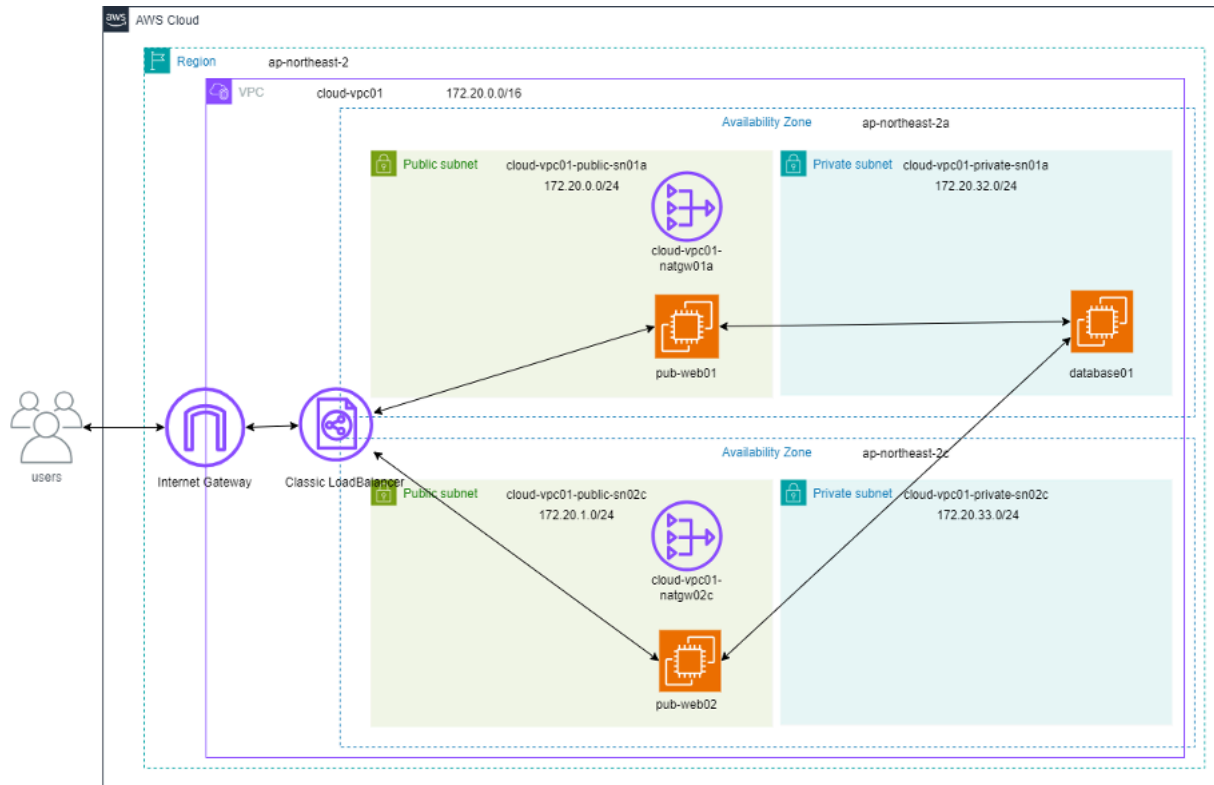
본 프로젝트는 Terraform과 Ansible을 활용하여 요구사항에 맞는 AWS 클라우드 리소스 아키텍처를 설계하고 구현합니다.

사용 환경 및 소프트웨어 버전

사용 목적	소프트웨어 / 환경	버전
운영체제	Ubuntu 22.04 LTS	Linux Kernel 6.8.0
클라우드 서비스	AWS	
인프라 관리	Terraform	1.12.0 (linux_amd
서버 구성 관리	Ansible	2.10.8
기타	Python	3.10.12

아키텍처

다음 그림과 같은 아키텍처를 구현합니다.



하나의 VPC 내에 2개의 웹 서버를 구축하며 두 웹 서버는 프라이빗 서브넷에 위치한 단일 DB 서버를 공유합니다.

DB 서버는 프라이빗 서브넷에 위치하고 있기 때문에 Ansible을 통해 서버 구성 자동화를 실행하기 위해서 퍼블릭 서브넷에 위치한 웹 서버를 점프 서버로 사용합니다.

외부 트래픽은 Classic 로드 밸런서를 통해 2개의 웹 서버로 분산되어 전달됩니다.

AWS 리소스

	이름	설명	가용 영역 / 서브넷	IPv4 CIDR
리전	ap-northeast-2			
VPC	cloud-vpc01			172.20.0.0/16
서브넷	cloud-vpc01-public-sn01a	퍼블릭 서브넷 1	ap-northeast-2a	172.20.0.0/24
	cloud-vpc01-public-sn02c	퍼블릭 서브넷 2	ap-northeast-2c	172.20.1.0.24
	cloud-vpc01-private-sn01a	프라이빗 서브넷 1	ap-northeast-2a	172.20.32.0/24

	cloud-vpc01-private-sn02c	프라이빗 서브넷 2	ap-northeast-2c	172.20.33.0/24
탄력적 IP	-	2개 생성 후 NAT 게이트웨이에 하나씩 부여		
NAT 게이트웨이	cloud-vpc01-natgw01a		ap-northeast-2a	
	cloud-vpc01-natgw02c		ap-northeast-2c	
인터넷 게이트웨이	cloud-vpc01-igw	cloud-vpc01과 연결		
라우팅 테이블	rt-public	퍼블릭 서브넷 1,2의 모든 트래픽 cloud-vpc01-igw로 라우팅		
	rt-private-sn01a	프라이빗 서브넷 1의 모든 트래픽 cloud-vpc01-natgw01a로 라우팅		
	rt-private-sn02c	프라이빗 서브넷 2의 모든 트래픽 cloud-vpc01-natgw02c로 라우팅		
EC2	pub-web01	NginX로 웹 서버 구성	cloud-vpc01-public-sn01a	
	pub-web02	NginX로 웹 서버 구성	cloud-vpc01-public-sn02c	
	database1	MySQL로 DB 서버 구성	cloud-vpc01-private-sn01a	
키 페어	my-key	모든 EC2 인스턴스에 사용		
보안 그룹	web-sg	인바운드 22, 80 포트 모두 허용 아웃바운드 모두 허용		
	db-sg	인바운드 3306 포트 모두 허용		

		아웃바운드 모두 허용		
Classic LoadBalancer	cloud-clb			

프로젝트 내용

이 장에서는 tf-project라는 이름의 루트 디렉토리를 생성하고 Terraform과 Ansible 관련 파일들을 각각의 디렉토리로 분리하여 작성합니다. 또한, 각 도구를 실행할 수 있는 쉘 스크립트를 통해 인프라를 자동으로 구성하고 서버를 구현합니다.

디렉토리 구조

먼저 프로젝트 루트 디렉토리를 생성하고 Terraform과 Ansible 디렉토리를 만들어줍니다.

```
$ mkdir -p tf-project/terraform
$ mkdir -p tf-project/ansible
$ cd tf-project
```

디렉토리 구조는 다음과 같습니다.

```
tf-project/
├── terraform/      # Terraform 파일 저장
├── ansible/        # Ansible 파일 저장
├── run_terraform.sh # Terraform 실행 스크립트
└── run_ansible.sh  # Ansible 실행 스크립트
```

Terraform 파일 작성

AWS 리소스를 생성하기 위한 Terraform 파일을 작성합니다. 다음과 같은 순서로 진행됩니다.

1. 프로바이더 설정
2. VPC 및 네트워크 구성
3. EC2 인스턴스 생성
4. 로드 밸런서 생성 및 EC2 연결
5. 출력값 작성

코드 설명 시에 Terraform 코드 내에서의 리소스 식별자와 실제 AWS 리소스에 부여한 이름 태그 값이 혼동될 수 있어서 구분이 필요합니다. 리소스를 참조하는 경우에는 Terraform 식별자를 그대로 사용하고 이름 태그 값은 따옴표(" ")로 감싸 표기하였습니다.

1. 프로바이더 설정

프로바이더는 Terraform이 외부 인프라와 상호작용할 수 있도록 해주는 플러그인입니다. AWS와 상호작용하기 위해 aws 프로바이더를 설정합니다.

- 프로바이더 생성

provider.tf

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 4.0"
    }
  }
}

provider "aws" {
  profile = "default"
  region = "ap-northeast-2"
}
```

`terraform init` 명령어를 실행하면 현재 디렉토리에 `.terraform/`, `.terraform.lock.hcl` 파일이 생성되는데 이 파일들은 프로바이더에서 설정한 내용을 기반으로 작성됩니다.

`.terraform/` 디렉토리는 프로바이더 플러그인이 저장되는 디렉토리이고 `.terraform.lock.hcl` 파일은 프로바이더 버전을 고정하기 위한 파일입니다.

2. VPC 및 네트워크 구성

현재 아키텍처의 VPC는 총 4개의 서브넷으로 다음과 같이 구성되어 있습니다.

- 가용 영역 ap-northeast-2a: 퍼블릭 서브넷 1개, 프라이빗 서브넷 1개
- 가용 영역 ap-northeast-2c: 퍼블릭 서브넷 1개, 프라이빗 서브넷 1개

프라이빗 서브넷 내 리소스가 외부와 아웃바운드 통신을 할 수 있도록 각 가용 영역의 퍼블릭 서브넷에 NAT 게이트웨이를 생성하고 해당 가용 영역의 프라이빗 서브넷의 트래픽이 NAT 게이트웨이를 통해 나갈 수 있도록 라우팅 테이블을 생성해 라우팅합니다.

퍼블릭 서브넷 내 리소스가 외부와 양방향 통신을 할 수 있도록 인터넷 게이트웨이를 생성하여 VPC에 연결하고 퍼블릭 서브넷의 트래픽이 인터넷 게이트웨이를 통해 나갈 수 있도록 라우팅 테이블을 생성해 라우팅합니다. 참고로 하나의 VPC에는 하나의 인터넷 게이트웨이만 연결될 수 있습니다.

파일 작성 순서

1. VPC 생성
2. 서브넷 생성
3. NAT 게이트웨이 생성
4. 인터넷 게이트웨이 생성
5. 라우팅 테이블 생성

Terraform은 전체 파일을 한 번에 읽고 의존성을 파악하여 실행 순서를 결정하기 때문에 파일 작성 순서는 크게 중요하지 않지만 작성의 편의성을 위해 생성 흐름에 따라 순서대로 구성합니다.

- **VPC 생성**

프로젝트의 기본 네트워크 환경이 될 `cloud-vpc01` VPC를 생성합니다.

vpc.tf

```
resource "aws_vpc" "cloud_vpc01" {  
  cidr_block = "172.20.0.0/16"  
  
  tags = {  
    Name = "cloud-vpc01"  
  }  
}
```

- aws_vpc.cloud_vpc01
이름 : "cloud-vpc01"
IPv4 CIDR 블록 : 172.20.0.0/16

- **서브넷 생성**

subnet.tf

```
resource "aws_subnet" "vpc01_public01a" {
  vpc_id          = aws_vpc.cloud_vpc01.id
  cidr_block      = "172.20.0.0/24"
  availability_zone = "ap-northeast-2a"
  map_public_ip_on_launch = true

  tags = {
    Name = "cloud-vpc01-public-sn01a"
  }
}
```

- aws_subnet.vpc01_public01a
 - 이름 : "cloud-vpc01-public-sn01a"
 - vpc : aws_vpc.cloud_vpc01
 - IPv4 CIDR 블록 : 172.20.0.0/24
 - 가용 영역 : ap-northeast-2a
 - 퍼블릭 IP 할당 : true

```
resource "aws_subnet" "vpc01_public02c" {
  vpc_id          = aws_vpc.cloud_vpc01.id
  cidr_block      = "172.20.1.0/24"
  availability_zone = "ap-northeast-2c"
  map_public_ip_on_launch = true

  tags = {
    Name = "cloud-vpc01-public-sn02c"
  }
}
```

- aws_subnet.vpc01_public02c
 - 이름 : "cloud-vpc01-public-sn02c"
 - vpc : aws_vpc.cloud_vpc01
 - IPv4 CIDR 블록 : 172.20.1.0/24
 - 가용 영역 : ap-northeast-2c
 - 퍼블릭 IP 할당 : true


```
resource "aws_subnet" "vpc01_private01a" {
  vpc_id      = aws_vpc.cloud_vpc01.id
  cidr_block  = "172.20.32.0/24"
  availability_zone = "ap-northeast-2a"

  tags = {
    Name = "cloud-vpc01-private-sn01a"
  }
}
```

- aws_subnet.vpc01_private01a
 - 이름 : "cloud-vpc01-private-sn01a"
 - vpc : aws_vpc.cloud_vpc01
 - IPv4 CIDR 블록 : 172.20.32.0/24
 - 가용 영역 : ap-northeast-2a
 - 퍼블릭 IP 할당 : false (기본값)

```
resource "aws_subnet" "vpc01_private02c" {
  vpc_id      = aws_vpc.cloud_vpc01.id
  cidr_block  = "172.20.33.0/24"
  availability_zone = "ap-northeast-2c"

  tags = {
    Name = "cloud-vpc01-private-sn02c"
  }
}
```

- aws_subnet.vpc01_private02c
 - 이름 : "cloud-vpc01-private-sn02c"
 - vpc : aws_vpc.cloud_vpc01
 - IPv4 CIDR 블록 : 172.20.33.0/24
 - 가용 영역 : ap-northeast-2c
 - 퍼블릭 IP 할당 : false (기본값)

cloud-vpc01에 4개의 서브넷을 생성합니다. ap-northeast-2a 가용 영역에 퍼블릭 서브넷과 프라이빗 서브넷 각 1개씩, ap-northeast-2c 가용 영역에도 퍼블릭 서브

넷과 프라이빗 서브넷 각 1개씩을 배치합니다.

가용 영역을 나누어 서브넷을 배치하는 이유는 하나의 가용 영역에 장애가 발생하더라도 다른 가용 영역의 서브넷을 통해 서비스를 지속할 수 있도록 하기 위함입니다.

- **NAT 게이트웨이 생성**

각 가용 영역에 NAT 게이트웨이를 1개씩 생성합니다. 현재 아키텍처에서는 두 개의 가용 영역을 사용하므로 총 2개의 NAT 게이트웨이를 생성합니다.

natgw.tf

```
resource "aws_eip" "vpc01_natgw01a" {}

resource "aws_eip" "vpc01_natgw02c" {}
```

- aws_eip.vpc01_natgw01a
- aws_eip.vpc01_natgw02c

```
resource "aws_nat_gateway" "vpc01_natgw01a" {
  subnet_id    = aws_subnet.vpc01_public01a.id
  allocation_id = aws_eip.vpc01_natgw01a.id

  tags = {
    Name : "cloud-vpc01-natgw01a"
  }
}

resource "aws_nat_gateway" "vpc01_natgw02c" {
  subnet_id    = aws_subnet.vpc01_public02c.id
  allocation_id = aws_eip.vpc01_natgw02c.id

  tags = {
    Name : "cloud-vpc01-natgw02c"
  }
}
```

- aws_nat_gateway.vpc01_natgw01a
 - 이름 : "cloud-vpc01-natgw01a"
 - 서브넷 : aws_subnet.vpc01_private01a
 - 탄력적 IP 주소 : aws_eip.vpc01_natgw01a
- aws_nat_gateway.vpc01_natgw02c
 - 이름 : "cloud-vpc01-natgw02c"
 - 서브넷 : aws_subnet.vpc01_private02c
 - 탄력적 IP 주소 : aws_eip.vpc01_natgw02c

NAT 게이트웨이는 각 가용 영역의 퍼블릭 서브넷 내에 위치하며 고정된 공인 IP 주소를 사용하기 위해 탄력적 IP 주소를 생성해 연결합니다.

- **인터넷 게이트웨이 생성**

`cloud-vpc01`에 연결할 인터넷 게이트웨이를 생성합니다.

igw.tf

```
resource "aws_internet_gateway" "vpc01_igw" {
  vpc_id = aws_vpc.cloud_vpc01.id

  tags = {
    Name = "cloud-vpc01-igw"
  }
}
```

- `aws_internet_gateway.vpc01_igw`
 - `이름` : "cloud-vpc01-igw"
 - `vpc` : `aws_vpc.cloud_vpc01`

- **라우팅 테이블 생성**

각 서브넷에 알맞은 게이트웨이를 연결해주기 위해 라우팅 테이블을 생성합니다.

route_table.tf

```
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.cloud_vpc01.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.vpc01_igw.id
  }

  tags = {
    Name = "rt-public"
  }
}

resource "aws_route_table_association" "public01a" {
  subnet_id    = aws_subnet.vpc01_public01a.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "public02c" {
  subnet_id    = aws_subnet.vpc01_public02c.id
  route_table_id = aws_route_table.public.id
}
```

- `aws_route_table.public`
 - `이름` : "rt-public"
 - `vpc` : `aws_vpc.cloud_vpc01`
 - `라우팅` : 모든 트래픽(0.0.0.0/0) → `aws_internet_gateway.vpc01_igw`
- `aws_route_table_association.public01a`
 - `서브넷` : `aws_subnet.vpc01_public01a`
 - `라우팅 테이블` : `aws_route_table.public`
→ public 라우팅 테이블을 서브넷 `vpc01_public01a`에서 적용
- `aws_route_table_association.public02c`
 - `서브넷` : `aws_subnet.vpc01_public02c`
 - `라우팅 테이블` : `aws_route_table.public`
→ public 라우팅 테이블을 서브넷 `vpc01_public02c`에서 적용

```

resource "aws_route_table" "private01a" {
  vpc_id = aws_vpc.cloud_vpc01.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_nat_gateway.vpc01_natgw01a.id
  }

  tags = {
    Name = "rt-private-sn01a"
  }
}

resource "aws_route_table_association" "private01a" {
  subnet_id      = aws_subnet.vpc01_private01a.id
  route_table_id = aws_route_table.private01a.id
}

```

- aws_route_table.private01a
 - 이름 : "rt-private-sn01a"
 - vpc : aws_vpc.cloud_vpc01
 - 라우팅 : 모든 트래픽(0.0.0.0/0) → aws_nat_gateway.vpc01_natgw01a
- aws_route_table_association.private01a
 - 서브넷 : aws_subnet.vpc01_private01a
 - 라우팅 테이블 : aws_route_table.private01a
 - public 라우팅 테이블을 서브넷 vpc01_private01a에서 적용

```

resource "aws_route_table" "private02c" {
  vpc_id = aws_vpc.cloud_vpc01.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_nat_gateway.vpc01_natgw02c.id
  }

  tags = {

```

```

    Name = "rt-private-sn02c"
  }
}

resource "aws_route_table_association" "private02c" {
  subnet_id      = aws_subnet.vpc01_private02c.id
  route_table_id = aws_route_table.private02c.id
}

```

- aws_route_table.private02c
 이름 : "rt-private-sn02c"
 vpc : aws_vpc.cloud-vpc01
 라우팅 : 모든 트래픽(0.0.0.0/0) → aws_nat_gateway.vpc01-natgw02c
- aws_route_table_association.private02c
 서브넷 : aws_subnet.vpc01_private02c
 라우팅 테이블 : aws_route_table.private02c
 → public 라우팅 테이블의 라우팅을 서브넷 vpc01_private02c에서 적용

3. EC2 인스턴스 생성

현재 아키텍처는 다음과 같은 EC2 인스턴스로 구성되어 있습니다:

- 가용 영역 ap-northeast-2a: 웹 서버 1대(퍼블릭 서브넷), DB 서버 1대(프라이빗 서브넷)
- 가용 영역 ap-northeast-2c: 웹 서버 1대(퍼블릭 서브넷)

웹 서버와 DB 서버는 각각의 목적에 맞게 설정된 보안 그룹을 사용하며 필요한 포트의 트래픽만 허용하도록 구성합니다.

또한, 모든 인스턴스는 동일한 키 페어를 사용하여 SSH 접근이 가능하도록 설정합니다.

파일 작성 순서

1. 키 페어 생성
2. 보안 그룹 생성

3. EC2 인스턴스 생성

- 키 페어 생성

key-pair.tf

```
resource "tls_private_key" "my_key" {
  algorithm = "RSA"
  rsa_bits  = "4096"
}

resource "local_file" "my_key_pem" {
  filename      = "my-key.pem"
  content       = tls_private_key.my_key.private_key_pem
  file_permission = "0400"
}

resource "aws_key_pair" "my_key_openssh" {
  key_name   = "my-key"
  public_key = tls_private_key.my_key.public_key_openssh

  tags = {
    Name = "my-key"
  }
}
```

- `tls_private_key.my_key`
 - 알고리즘 : RSA
 - RSA 비트 : 4096
 - 키 페어를 생성합니다.
- `local_file.my_key_pem`
 - 파일 이름 : my-key.pem
 - 콘텐츠(저장할 콘텐츠) : `tls_private_key.my_key` 리소스에서 생성된 개인 키
 - 파일 권한 : 0400
 - 개인 키 파일을 로컬에 저장합니다.
- `aws_key_pair.my_key_openssh`
 - 이름 : "my-key"
 - 키 이름: my-key
 - public_key : `tls_private_key.my_key.public_key_openssh`
 - AWS에 공개 키를 등록하여 EC2 인스턴스 생성 시 자동으로 `.ssh/authorized_keys`에 삽입되도록 합니다.

EC2 인스턴스에 SSH로 접속하기 위해서 `tls_private_key` 리소스를 사용하여 SSH 키 페어(개인 키와 공개 키)를 생성합니다.

개인 키는 `local_file` 리소스를 통해 `my_key.pem` 파일로 로컬에 저장되며 EC2 인스턴스에 SSH 접속을 할 때 사용됩니다.

공개 키는 `aws_key_pair` 리소스를 통해 AWS에 등록됩니다. 이제 EC2 인스턴스를 생성할 때 해당 키를 참조하면 공개 키가 인스턴스의 `~/.ssh/authorized_keys` 파일에 자동으로 등록됩니다. 해당 인스턴스에는 로컬에 저장된 개인 키를 사용해 SSH 접속할 수 있습니다.

- 보안 그룹 생성

security_group.tf

```
resource "aws_security_group" "web" {
  name      = "web-sg"
  description = "for web server"
  vpc_id    = aws_vpc.cloud_vpc01.id

  ingress {
    description = "SSH From anywhere"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTP From anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "web-sg"
  }
}
```

- aws_security_group.web
 - 이름 : "web-sg"
 - VPC : aws_vpc.cloud_vpc01

인바운드 규칙 1 :

- **프로토콜** : tcp
- **포트** : 22
- **허용 대상 IP** : 0.0.0.0/0(모든 IP 주소)

인바운드 규칙 2 :

- **프로토콜** : tcp
- **포트** : 80
- **허용 대상 IP** : 0.0.0.0/0(모든 IP 주소)

아웃바운드 규칙 :

- **프로토콜** : -1(모든 프로토콜)
- **포트** : 0(모든 포트)
- **허용 대상 IP** : 0.0.0.0/0(모든 IP 주소)

```
resource "aws_security_group" "db" {
  name      = "db-sg"
  description = "for db server"
  vpc_id    = aws_vpc.cloud_vpc01.id

  ingress {
    description = "SSH From anywhere"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "MySQL From web security group"
    from_port   = 3306
    to_port     = 3306
    protocol    = "tcp"
    security_groups = [aws_security_group.web.id]
  }
}
```

```

egress {
  from_port = 0
  to_port   = 0
  protocol  = -1
  cidr_blocks = ["0.0.0.0/0"]
}

tags = {
  Name = "db-sg"
}
}

```

- aws_security_group.db

이름 : "db-sg"

VPC : aws_vpc.cloud_vpc01

인바운드 규칙 1 :

- 프로토콜 : tcp
- 포트 : 22
- 허용 대상 IP : 0.0.0.0/0(모든 IP 주소)

인바운드 규칙 2 :

- 프로토콜 : tcp
- 포트 : 3306
- 허용 대상 보안 그룹 : aws_security_group.web
→ DB 서버는 웹 서버에서만 접속할 수 있도록 합니다.

아웃바운드 규칙 :

- 프로토콜 : -1(모든 프로토콜)
- 포트 : 0(모든 포트)
- 허용 대상 IP : 0.0.0.0/0(모든 IP 주소)

웹 서버에서 사용하는 보안 그룹에는 모든 IP 주소로부터 22번 포트와 80번 포트로 들어오는 트래픽을 허용합니다. 22번 포트는 SSH 접속할 때 사용되고 80번 포트는 HTTP에서 사용합니다.

DB 서버에서 사용하는 보안 그룹에는 모든 IP 주소로부터 22번 포트와 웹 보안 그룹을 사용하는 인스턴스에서 3306번 포트에 들어오는 트래픽을 허용합니다. 3306 포트는 MySQL에서 사용합니다.

두 보안 그룹 모두 나가는 트래픽은 모든 포트, 모든 목적지를 허용하도록 합니다.

이 프로젝트에서는 단일 포트만 개방했지만 `from_port` 와 `to_port` 를 조절하여 허용할 포트 범위를 설정할 수도 있습니다. 예를 들어 `from_port` 를 3000, `to_port` 를 3002 로 설정하면 3000, 3001, 3002 포트를 허용합니다.

- **EC2 인스턴스 생성**

두 웹 서버 인스턴스는 Amazon Linux 2023 AMI(Amazon Machine Image)를 사용하고 DB 서버 인스턴스는 Ubuntu 22.04 AMI를 사용합니다.

AMI ID를 지정하여 설정하는 방법도 있지만 가장 최신의 AMI를 사용하기 위해 SSM(Systems Manager) 파라미터를 사용합니다.

instance.tf

```
data "aws_ssm_parameter" "amazon_linux_2023_ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

data "aws_ssm_parameter" "ubuntu_22_04_ami" {
  name = "/aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebs-gp2/ami-id"
}
```

- `aws_ssm_parameter.amazon_linux_2023_ami`
→ 웹 서버 인스턴스에서 사용할 Amazon Linux 2023 AMI 최신 이미지의 파라미터를 가져옵니다.
- `aws_ssm_parameter.ubuntu_22_04_ami`
→ DB 서버 인스턴스에서 사용할 Ubuntu 22.04 AMI 최신 이미지의 파라미터를 가져옵니다.

```
resource "aws_instance" "pub_web01" {
  ami              = data.aws_ssm_parameter.amazon_linux_2023_ami.value
  associate_public_ip_address = true
  instance_type    = "t2.micro"
  key_name         = aws_key_pair.my_key_openssh.key_name
  subnet_id       = aws_subnet.vpc01_public01a.id

  vpc_security_group_ids = [aws_security_group.web.id]

  tags = {
    Name = "pub-web01"
  }
}
```

- `aws_instance.pub_web01`
`이름`: "pub-web01"

AMI : aws_ssm_parameter.amazon_linux_2023_ami

퍼블릭 IP 할당 : true

인스턴스 타입 : t2.micro

키 이름(공개 키 참조) : aws_key_pair.my_key_openssh

서브넷 : aws_subnet.vpc01_public01a

보안 그룹 : aws_security_group.web

```
resource "aws_instance" "pub_web02" {
  ami                = data.aws_ssm_parameter.amazon_linux_2023_ami.value
  associate_public_ip_address = true
  instance_type      = "t2.micro"
  key_name            = aws_key_pair.my_key_openssh.key_name
  subnet_id          = aws_subnet.vpc01_public02c.id

  vpc_security_group_ids = [aws_security_group.web.id]

  tags = {
    Name = "pub-web02"
  }
}
```

- aws_instance.pub_web02

이름 : "pub-web02"

AMI : aws_ssm_parameter.amazon_linux_2023_ami

퍼블릭 IP 할당 : true

인스턴스 타입 : t2.micro

키 이름(공개 키 참조) : aws_key_pair.my_key_openssh

서브넷 : aws_subnet.vpc01_public02c

보안 그룹 : aws_security_group.web

```
resource "aws_instance" "database01" {
  ami                = data.aws_ssm_parameter.ubuntu_22_04_ami.value
  instance_type      = "t2.micro"
  key_name            = aws_key_pair.my_key_openssh.key_name
  subnet_id          = aws_subnet.vpc01_private01a.id
}
```



```

vpc_security_group_ids = [aws_security_group.db.id]

tags = {
  Name = "database01"
}
}

```

- aws_instance.database01
 - 이름 : "database01"
 - AMI : aws_ssm_parameter.ubuntu_22_04_ami
 - 퍼블릭 IP 할당 : true
 - 인스턴스 타입 : t2.micro
 - 키 이름(공개 키 참조) : aws_key_pair.my_key_openssh
 - 서브넷 : aws_subnet.vpc01_private01a
 - 보안 그룹 : aws_security_group.db

4. 로드 밸런서 생성 및 EC2 연결

현재 아키텍처에서는 Classic 로드 밸런서(ELB)를 사용하여 외부에서 들어오는 80 포트 요청을 웹 서버 인스턴스 2대의 80 포트에 균등하게 분산하여 전송합니다.

lb.tf

```
resource "aws_elb" "cloud_clb" {
  name = "cloud-clb"
  security_groups = [aws_security_group.web.id]
  subnets = [
    aws_subnet.vpc01_public01a.id,
    aws_subnet.vpc01_public02c.id
  ]
  listener {
    instance_port    = 80
    instance_protocol = "HTTP"
    lb_port          = 80
    lb_protocol       = "HTTP"
  }
  health_check {
    healthy_threshold    = 10
    unhealthy_threshold = 2
    target                = "HTTP:80/"
    interval              = 5
    timeout               = 2
  }

  tags = {
    Name = "cloud-clb"
  }
}
```

- aws_elb.cloud_clb

이름 : "cloud-clb"

보안 그룹 : aws_security_group.web

서브넷 : aws_subnet.vpc01_public01a, aws_subnet.vpc01_public02c

리스너 :

- 로드 밸런서의 HTTP 80 → 인스턴스의 HTTP 80으로 전송

헬스 체크 :

- 정상 임계값 : 10

- 비정상 임계값 : 2
- 체크할 타겟 : HTTP 프로토콜 / 80 포트 / 루트 디렉토리
- 인터벌 : 5
- 타임아웃 : 2

```
resource "aws_elb_attachment" "web01" {
  elb = aws_elb.cloud_clb.id
  instance = aws_instance.pub_web01.id
}

resource "aws_elb_attachment" "web02" {
  elb = aws_elb.cloud_clb.id
  instance = aws_instance.pub_web02.id
}
```

- aws_elb_attachment.web01
→ aws_elb.cloud_clb에 aws_instance.pub_web01 연결
- aws_elb_attachment.web02
→ aws_elb.cloud_clb에 aws_instance.pub_web02 연결

5. 출력값 작성

서버 구성 시 각 인스턴스를 Ansible 관리 노드에 등록하려면 인스턴스의 공인 IP나 사설 IP 주소가 필요합니다.

이 정보를 얻기 위해 Terraform의 output 블록을 사용하여 인스턴스의 IP 주소를 출력값으로 설정합니다.

outputs.tf

```
output "web01_pub_ip" {
  value = aws_instance.pub_web01.public_ip
}

output "web02_pub_ip" {
  value = aws_instance.pub_web02.public_ip
}

output "db01_priv_ip" {
  value = aws_instance.database01.private_ip
}

output "elb_dns_addr" {
  value = aws_elb.cloud_clb.dns_name
}
```

Ansible 파일 작성

서버 구성을 위한 Ansible 파일을 작성합니다.

웹 서버 구성을 자동화하는 nginx 역할과 DB 서버 구성을 자동화하는 mysql 역할로 나누어서 작성됩니다. ansible 디렉토리의 구조는 다음과 같습니다.

```
ansible/
├── inventory/
│   ├── hosts.ini          # 관리 노드 설정 파일
│   └── group_vars/        # 호스트 그룹별 변수 파일
│       ├── all.yml
│       ├── webservers.yml
│       └── dbservers.yml
├── roles/
│   ├── nginx/             # 웹 서버 구성 역할
│   └── mysql/              # DB 서버 구성 역할
```

```
|— ansible.cfg      # Ansible 설정 파일
|— site.yml        # 메인 플레이북
```

파일 작성 순서

1. Ansible 설정 파일 작성
2. 인벤토리 파일 작성
3. mysql 역할 작성
4. nginx 역할 작성
5. site.yml 파일 작성

1. Ansible 설정 파일 작성

ansible.cfg

```
[defaults]
inventory=../inventory/hosts.ini
private_key_file=../terraform/my-key.pem
host_key_checking=False

[privilege_escalation]
become=true
become_method=sudo
become_user=root

[ssh_connection]
ssh_args = -o StrictHostKeyChecking=no
```

인벤토리 파일, 관리 노드에 SSH 접속 시 필요한 개인 키 파일을 지정합니다. 최초 SSH 연결 시 호스트 키 확인 대화형 프롬프트를 띄우지 않겠다는 의미로 `host_key_checking` 을 `False` 로 설정하고 SSH 연결 인자를 설정합니다.

2. 인벤토리 파일 작성

inventory 디렉토리에는 그룹 호스트별 변수와 Ansible에서 다룰 호스트를 정의한 파일을 작성합니다.

hosts.ini

```
[webservers]
web01 ansible_host=web01_pub_ip
web02 ansible_host=web02_pub_ip

[dbservers]
db01 ansible_host=db01_priv_ip ansible_ssh_common_args='-o ProxyCommand="ssh -i ../terraform/my-key.pem -W %h:%p -q ec2-user@web01_pub_ip"'
```

이 파일은 Terraform 스크립트 실행 시 자동으로 생성되는 인벤토리 파일입니다. 작성되는 IP 주소는 Terraform의 출력값을 기반으로 자동으로 채워집니다. DB 서버는 웹 서버를 점프 서버로 사용하여 SSH 접속을 해야 하므로 `ansible_ssh_common_args` 변수를 통해 ProxyCommand 설정이 포함됩니다.

group_vars/
all.yml

```
mysql_root_password: "admin123"  
app_db: "cccr_db"  
app_user: "cccr"  
app_password: "cccr123"
```

→ 웹 서버와 DB 서버에서 모두 사용하는 변수를 정의합니다.

dbservers.yml

```
ansible_user: ubuntu
```

→ DB 서버의 OS는 Ubuntu이기 때문에 사용자명을 ubuntu로 설정합니다.

webservers.yml

```
ansible_user: ec2-user
```

→ 웹 서버의 OS는 Amazon Linux 2023이어서 사용자명을 ec2-user로 설정합니다.

3. mysql 역할 작성

DB 서버를 구성하는 작업들을 `roles/mysql/tasks` 디렉토리에 작성합니다.

main.yml

```
- name: Install mysql package
  apt:
    name: "{{ item }}"
    state: present
  loop:
    - python3-pymysql
    - mysql-server
```

관리 노드에서 사용하는 모듈인 python과 DB 서버 구성을 위해 필요한 mysql 패키지를 설치합니다.

```
- name: Start and enable MySQL
  service:
    name: mysql
    state: started
    enabled: true
```

MySQL 시스템 데몬을 시작하고 재부팅 시에도 실행 상태로 유지되도록 합니다.

```
- name: Set mysql root password
  mysql_user:
    name: root
    host: localhost
    password: "{{ mysql_root_password }}"
    check_implicit_admin: yes
    login_unix_socket: /var/run/mysqld/mysqld.sock
    state: present
    ignore_errors: yes
```

MySQL 서버의 루트 사용자 비밀번호를 설정합니다.

이미 비밀번호가 설정되어 에러가 발생할 경우에도 플레이북 실행을 멈추지 않도록 `ignore_errors: yes` 로 설정합니다.

```
- name: Create app db
  mysql_db:
    name: "{{ app_db }}"
```



```
state: present
login_user: root
login_password: "{{ mysql_root_password }}"
```

웹 서버에서 사용할 데이터베이스를 생성합니다.

```
- name: Allow user to connect from anywhere
mysql_user:
  name: "{{ app_user }}"
  password: "{{ app_password }}"
  priv: "{{ app_db }}.*:ALL"
  host: "%"
  state: present
  login_user: root
  login_password: "{{ mysql_root_password }}"
```

웹 서버에서 사용할 사용자를 생성합니다.

4. nginx 역할 작성

웹 서버를 구성하는 작업들을 `roles/nginx/tasks` 디렉토리에 작성합니다. 작업에서 사용될 템플릿은 `roles/nginx/templates` 디렉토리에 작성합니다.

templates/index.html.j2

```
<!DOCTYPE html>
<html>
<head>
  <title>Welcome!</title>
</head>
<body>
  <h1>This is {{ inventory_hostname }}</h1>
</body>
</html>
```

이 파일은 웹 서버의 `index.html` 파일을 위한 템플릿입니다. `inventory_hostname` 변수를 사용하여 현재 서버의 이름을 작성해 각 웹 서버를 구분할 수 있게 되어 있습니다.

main.yml

```
- name: Install nginx package
  dnf:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
```

웹 서버 구성을 위해 필요한 NginX 패키지를 설치합니다.

```
- name: Deploy index.html
  template:
    src: index.html.j2
    dest: /usr/share/nginx/html/index.html
    owner: root
    group: root
    mode: '0644'
```

아까 작성한 템플릿 파일을 웹 서버의 루트 디렉토리로 옮기고 권한 설정을 합니다.

```
- name: Restart and enabled nginx
  service:
    name: nginx
    state: restarted
    enabled: true
```

NginX 시스템 데몬을 시작하고 재부팅 시에도 실행 상태로 유지되도록 합니다.

5. site.yml 파일 작성

작성한 역할에 맞게 작업을 실행할 `site.yml` 플레이북을 작성합니다.

site.yml

```
---  
- name: Configure db server  
  hosts: dbservers  
  roles:  
    - mysql  
  
- name: Configure web server  
  hosts: webservers  
  roles:  
    - nginx
```

dbservers 호스트 그룹은 mysql 역할을 실행하고 webservers 호스트 그룹의 호스트들은 nginx 역할을 실행하는 플레이북입니다.

자동화 스크립트 작성

아키텍처 구현을 위한 모든 Terraform 파일과 Ansible 파일 작성이 완료되었습니다.

이제 이 파일들을 실행할 자동화 스크립트(`run_terraform.sh`, `run_ansible.sh`)를 작성합니다.

run_terraform.sh

```
#!/bin/bash

set -e

TERRAFORM_DIR="./terraform"
ANSIBLE_INVENTORY_FILE="../ansible/inventory/hosts.ini"

# 터미널에 색상 출력용 변수
GREEN='\033[0;32m'
NC='\033[0m' # No Color

# Terraform
cd "$TERRAFORM_DIR"
echo -e "${GREEN}🔧 Initializing Terraform...${NC}"
terraform init

echo -e "${GREEN}🧠 Planning Terraform changes...${NC}"
terraform plan

echo -e "${GREEN}🚀 Applying Terraform changes...${NC}"
terraform apply -auto-approve

echo -e "${GREEN}💾 Saving output to output.txt...${NC}"
terraform output > output.txt

# Terraform 출력값
web01=$(terraform output -raw web01_pub_ip)
web02=$(terraform output -raw web02_pub_ip)
db01=$(terraform output -raw db01_priv_ip)

# ansible/inventory/hosts.ini 파일 작성
echo "[webservers]" > "$ANSIBLE_INVENTORY_FILE"
echo "web01 ansible_host=$web01" >> "$ANSIBLE_INVENTORY_FILE"
echo "web02 ansible_host=$web02" >> "$ANSIBLE_INVENTORY_FILE"
echo "" >> "$ANSIBLE_INVENTORY_FILE"
```

```

echo "[dbservers]" >> "$ANSIBLE_INVENTORY_FILE"
echo "db01 ansible_host=$db01 ansible_ssh_common_args='-o ProxyC
ommand=\"ssh -o StrictHostKeyChecking=no -i ../terraform/my-key.pe
m -W %h:%p -q ec2-user@$web01\"'" >> "$ANSIBLE_INVENTORY_FI
LE"

cd -

echo -e "${GREEN}✅ Done!${NC}"

```

run_ansible.sh

```

#!/bin/bash

set -e

ANSIBLE_DIR="./ansible"

# 터미널에 색상 출력용 변수
GREEN='\033[0;32m'
NC='\033[0m' # No Color

# Ansible
cd "$ANSIBLE_DIR"
echo -e "${GREEN}🚀 Running playbook site.yml..${NC}"
ansible-playbook site.yml

```

배포 및 검증

초기 설정

terraform을 사용하기 위한 초기 설정을 진행합니다.

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o  
"awscliv2.zip"  
$ unzip awscliv2.zip  
$ sudo ./aws/install  
$ aws --version # aws 설치 확인
```

AWS 패키지를 설치합니다.

AWS 콘솔에 접속해 프로젝트에서 사용할 AMI 사용자를 생성하고
AdministratorAccess, IAMFullAccess, PowerUserAccess 권한 정책을 추가합니
다.

그리고 해당 사용자의 액세스 키를 생성합니다.

```
$ aws configure  
AWS Access Key ID [None]: 액세스 키  
AWS Secret Access Key [None]: 비밀 액세스 키  
Default region name [None]: ap-northeast-2  
Default output format [None]:
```

생성한 액세스 키 정보를 입력합니다. 이 설정은 terraform에서 기본값으로 사용합니다.

```
$ sudo apt-get update && sudo apt-get install -y gnupg software-proper  
ties-common  
$ wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | su  
do tee /usr/share/keyrings/hashicorp-archive-keyring.gpg  
$ echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyrin  
g.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | su  
do tee /etc/apt/sources.list.d/hashicorp.list  
$ sudo apt-get update && sudo apt-get install terraform  
$ terraform -v
```

terraform 패키지를 설치합니다. 이제 terraform을 사용할 준비가 완료되었습니다.

자동화 스크립트 실행

```
$ ./run_terraform.sh  
$ ./run_ansible.sh
```

각 코드를 순서대로 실행하여 Terraform 파일과 Ansible 파일들을 실행합니다.

Aa 제목	🕒 문제 발생 구간
 <u>terraform init 실행 시 변화</u>	
 <u>AMI 정보 찾기</u>	
 <u>Ansible 점프 서버 설정</u>	